Zephyr Project

IT-Schulungen.com
Technology & Transformation

Praxis 5
GPIO-Emulation per Shell triggern

Einzelpegel, Puls und Burst reproduzierbar auslösen

3.10

Zielbild der Praxis

01	`Shell-Modul`	validiert Argumente und übersetzt Fehler für die Bedienung
02	`Stimulusadapter`	kapselt Pegel, Puls und Burst hinter einer klaren Schnittstelle
03	`GPIO-Emulator`	setzt unter native_sim den gewünschten Eingangszustand

Die Shell kennt den Kommandovertrag. Der Adapter besitzt Ablauf und Targetdetails.

Arbeitsauftrag in fünf Schritten

Das Source-Code-Projekt liefert Gerüst, Tests und markierte Ergänzungsstellen

01

Adapter trennen

öffentliche Stimulus-API definieren

02

input umsetzen

nur 0 oder 1 akzeptieren

03

pulse planen

aktive Flanke zeitgesteuert beenden

04

burst begrenzen

Anzahl und Abstand validieren

05

Bilanz prüfen

event stats nach jedem Stimulus lesen

Die Praxis endet mit reproduzierbaren Sequenzen und vorab definierten Zählerständen.

event input <0|1>

SHELL-HANDLER

```
int level;
if (argc != 2 || !parse_level(argv[1], &level)) {
    shell_error(sh, "value must be 0 or 1");
    return -EINVAL;
}

int rc = stimulus_set_level(level);
return shell_result(sh, rc);
```

01 Strikte Eingabe

Nur die vollständigen Werte 0 und 1 werden akzeptiert.

02 Klare Grenze

Der Handler ruft nur die öffentliche Stimulus-Schnittstelle auf.

Der native_sim-Adapter verwendet gpio_emul_input_set_dt() und übersetzt dessen Rückgabewert.

event pulse [dauer_ms]

PULSPLANUNG

```
int rc = stimulus_set_level(ACTIVE);
if (rc < 0) { return rc; }

return k_work_reschedule(
    &pulse_off_work,
    K_MSEC(duration_ms));
```

01 **Sofort aktiv**

Der Adapter stellt inaktiv sicher, setzt aktiv und kehrt ohne Wartezeit zurück.

02 **Verzögert inaktiv**

Ein delayable Work-Item beendet den Puls nach der validierten Dauer.

Eine Standarddauer ist dokumentiert. Ungültige oder zu große Werte werden vor dem Start abgelehnt.

event burst <anzahl> <abstand_ms>

SEQUENZZUSTAND

```
state.remaining = count;
state.gap_ms = gap_ms;
state.level = ACTIVE;

return k_work_reschedule(
    &burst_work,
    K_NO_WAIT);
```

01 Begrenzte Folge

Der Worker wechselt Pegel und reduziert den Restzähler.

02 Zeitgesteuert

Jeder weitere Schritt wird mit dem geprüften Abstand neu geplant.

Der Shell-Handler startet die Sequenz. Die Workqueue erzeugt sie schrittweise im Threadkontext.

Grenzen und laufende Sequenzen

KONFIGURATION

```
#define MAX_PULSE_MS        ...
#define MAX_BURST_COUNT     ...
#define MIN_BURST_GAP_MS    ...
#define MAX_BURST_GAP_MS    ...
```

01 Parametergrenzen

Benannte Konstanten begrenzen Dauer, Anzahl und Abstand.

02 Konfliktregel

Eine laufende Sequenz wird mit -EBUSY abgelehnt; Zustand bleibt unverändert.

03 Targetgrenze

Der Emulatoradapter ist nur für native_sim aktiv. Andere Targets melden die Funktion klar als nicht verfügbar.

Grenzen erscheinen in der Hilfe und werden mit Fehlerfällen getestet.

Prüfmatrix für Pegel, Puls und Burst

Einzelpegel	event input 0 und 1 setzen den erwarteten logischen Zustand
Puls	aktive Flanke entsteht; Rückkehr erfolgt nach der geplanten Dauer
Burst	Anzahl und Abstand führen zur vorab berechneten Ereignisbilanz
Ungültige Werte	Text, Vorzeichen, Überlauf und Bereichsfehler ändern keinen Zustand
Sequenz aktiv	neuer pulse oder burst liefert den dokumentierten Konfliktfehler

Nach jedem Fall zeigt event stats Callback-, Verarbeitungs- und Verlustzähler.

Abnahme der Shell-Stimuli

FUNKTION

Drei Kommandos

input setzt Einzelpegel

pulse erzeugt eine aktive Flanke

burst startet eine begrenzte Folge

ROBUSTHEIT

Definierte Fehlerwege

Argumente strikt geprüft

Grenzen eingehalten

Konflikte ohne Zustandsänderung

Fehler verständlich gemeldet

NACHWEIS

Zählerbilanz

Sollwerte vorab festgelegt

event stats nach jedem Test

Callback und Verarbeitung getrennt

Hardwarestation nicht erforderlich

Bestanden ist die Praxis, wenn Pegel, Puls und Burst reproduzierbar auslösbar und über Zähler nachweisbar sind.